Coverage map

Every sub-point of the phase material, and the page in the main PDF that explains it.

Use this page to prove nothing was dropped, and to jump straight to a topic when a reviewer asks a question you did not expect. Page numbers refer to WeWithU-Project-Review-Notes.pdf.

Phase 4 · Task	Points covered in the notes	Page
15 · Server bootstrap	what bootstrap means · dotenv · express() · CORS + credentials · express.json() · cookieParser() · route mounts · listen() · middleware order · the five closing questions	05–08
16 · Route files	what a route is · method/path/handlers · HTTP method table · prefix + router path · middleware in a route · six-step trace method · full endpoint map	08–10
17 · Auth middleware	JWT payload and signing · read cookie · missing token · verify · req.userId · next() · decision diagram · missing/invalid/valid table · 401 guidance · authn vs authz	10–12
18 · Current-user API	why the API exists · Axios call · Express middleware pass · router match · isAuth · controller · User model · response into Redux · eight-step sequence · security note	12–14
19 · Errors & status	400/401/403/404/500 with examples · actual vs expected table · why codes matter · per-controller audit · what to inspect in each controller	14–15
20 · Mongo connection	document/collection vocabulary · connectDb() · MONGODB_URL · startup timing · failure handling · debugging checklist	15–17
21 · Mongoose schemas	property cheat sheet · User · Interview + embedded questions · Payment · PlacementTest · how the models relate · ref semantics	17–19
22 · Data lifecycle	create (startPlacementTest) · read (find + sort + select) · update (findById → save) · fallback path · lifecycle diagram · Mongoose method table	19–21
23 · Validation	controller checks · schema rules · unique index and 11000 · five validation layers · nine gaps · why presence checks are not value checks	21–23
24 · Interview flow	five-step journey · setup state · FormData upload · generate-questions payload · answer submission · finish · what is stored	23–25
25 · AI generation	askAi options · JSON mode + extractJson · system/user prompts · evaluation prompt · bad-response comparison · AI vs database layers · prompt inventory	25–27
26 · Resume flows	file upload + Multer · PDF extraction · text parse · ATS check · guidance · four-flow comparison table	27–29
27 · Placement drive	five categories with counts and times · resume parse · start + credits · question generation per type · evaluation endpoints · final report · save · retrieve · lifecycle	29–31
28 · History & reports	two history GETs · _id to route to report · placement parity · three report stages · debugging order · loading vs error states	31–32
29 · Preparation pages	Guidance (API) · Resume Tips (API) · Company Prep (static) · Roadmap (static) · classification table for all pages	32–33

Plus Part V: findings register with severity, six end-to-end flows, answer keys, 20 viva questions, a two-day plan, demo script, glossary and the one-page cheat sheet (pages 34–38).

WHERE THE PHASE TEXT AND THE CODE DISAGREE

Topic	Phase material	This repository
Product name	ApnaCoach · apnacoach.onrender.com	WeWithU · wewithyou.onrender.com (same architecture)
Backend port	PORT || 6000, documented as 8000	PORT || 6000, .env.example 8000, frontend default 8000
DB connection	called inside the listen callback	same — traffic is accepted before Mongo is ready
Auth statuses	400 missing, 500 invalid	same, and the 401 recommendation is spelled out in Task 17

HOW TO USE THESE FOUR PAGES

- Page 1 is the index: prove coverage, then jump to a page in the main PDF when a question is unexpected.
- Page 2 is the backend card: bootstrap, auth, routing, status codes.
- Page 3 is the data card: schemas, lifecycles, validation layers, and the gaps worth naming.
- Page 4 is the feature card with the fifteen answer keys and the closing line to end on.

Backend card

Bootstrap, authentication, routing and the request lifecycle — compressed.

BOOTSTRAP ORDER (SERVER/INDEX.JS)

```
dotenv.config()                          -> process.env.*
const app = express()                    -> pipeline only, not listening
app.use(cors({ origin: allowedOrigins, credentials: true }))
app.use(express.json())                  -> req.body
app.use(cookieParser())                  -> req.cookies
app.use('/api/<name>', router) x8        -> auth user interview payment
                                            resume questions evaluate placement
app.listen(process.env.PORT || 6000, () => connectDb())
```

KNOWN CONFIGURATION MISMATCHES

- Backend default port 6000 · .env.example says 8000 · frontend defaults to 8000 → set PORT or VITE_API_URL.
- connectDb() runs inside the listen callback, so HTTP is accepted before Mongo is ready; failures are logged only.
- No global 404 handler and no Express error middleware in the bootstrap.

ISAUTH — FIVE STATEMENTS WORTH MEMORISING

- Reads req.cookies.token — available because cookieParser() ran earlier.
- No token → 400 today, conventionally 401.
- jwt.verify(token, JWT_SECRET) throws on malformed, tampered or expired tokens.
- Throw lands in catch → 500 today, conventionally 401; the second 400 branch is unreachable.
- Success sets req.userId = verifyToken.userId and calls next(); the controller never re-verifies.

STATUS CODES AS USED HERE

Code	Means	Seen in this repo
400	bad / incomplete request	missing fields, no token, low credits
401	identity not established	should replace the 400 and 500 auth cases
403	authenticated, not permitted	placement report ownership check
404	resource missing	no user, no interview, no report
409	conflict / duplicate	should be used for duplicate email (currently 500)
500	unexpected server failure	every controller catch; invalid JWT today

MIDDLEWARE — WHAT EACH ONE CONTRIBUTES

Middleware	Adds	Fails like this
cors({ origin, credentials })	browser permission + cookie allowance	browser blocks the response, no JS error detail
express.json()	req.body for JSON	req.body undefined → controller crash
cookieParser()	req.cookies	isAuth sees no token → 400
isAuth	req.userId after JWT verify	400 missing · 500 invalid (should be 401)
upload.single('resume')	req.file from multipart	400 'Resume required' / Multer limit error

THE EIGHT ROUTE MOUNTS

- /api/auth · google sign-in, logout · /api/user · current-user
- /api/interview · resume, generate-questions, submit-answer, finish, get-interview, report/:id
- /api/payment · order, verify · /api/placement · start, save, my-tests, report/:id
- /api/resume · parse, summarize, ats-check, guidance
- /api/questions · generate, technical, written, aptitude, softskills, project
- /api/evaluate · mcq, written, softskill, project, test-score, final-report, overall-score

TRACING ANY ENDPOINT IN SIX STEPS

- Search the client for axios.get / axios.post and note method + URL.
- Match the prefix in server/index.js (eight mounts).
- Open the router file and find the same method + remaining path.
- Read the middleware list left to right, starting with isAuth.
- Follow the controller into models (Mongoose) and services (askAi, razorpay).
- Confirm the response contract: status code + JSON shape the page actually reads.

Data card

Connection, four schemas, lifecycles and the validation layers.

CONNECTION

```
await mongoose.connect(process.env.MONGODB_URL)   // name is exactly MONGODB_URL
catch (e) { console.log(...) }                     // logged, not rethrown, no retry
// called from the app.listen callback -> DB may lag behind open HTTP port
```

SCHEMAS — WHAT A REVIEWER MAY ASK

Model / collection	Fields that matter	Rules to quote
User / users	name, email, credits	email unique + required; credits default 100; timestamps
Interview / interviews	userId, role, experience, mode, questions[], finalScore, status	mode enum technical|hr; status default Incompleted; questions embedded, not refs
Payment / payments	userId, planId, amount, credits, razorpayOrderId, razorpayPaymentId, status	status enum created|paid|failed, default created
PlacementTest / placementtests	userId, candidateName, targetRole, level, difficultyMode, resumeData, testScores[], allStrengths/Weaknesses, finalReport, status	status enum Incompleted|completed but default completed (contradicts startPlacementTest)

CREATE · READ · UPDATE IN PLACEMENT

```
create : PlacementTest.create({ ..., status: 'Incompleted' })  +  user.credits -= 50 ; user.save()
read   : PlacementTest.find({ userId: req.userId }).sort({ createdAt: -1 })
         .select('targetRole level finalReport createdAt status')
update : const test = await PlacementTest.findById(testId)
         test.testScores = ...; test.finalReport = ...; test.status = 'completed'; await test.save()
```

VALIDATION LAYERS — WHO OWNS WHAT

Layer	Owns	Example here
Frontend	experience, fewer wasted requests	credits < 50 blocks Start
Controller	business rules + ownership	required fields, credits, report ownership
Mongoose schema	types, required, enums, defaults	mode enum, credits default
MongoDB index	uniqueness	email unique → error 11000
isAuth	who the user is	JWT verification

MONGOOSE CALLS YOU SHOULD RECOGNISE

Call	Meaning	Where it appears
create(data)	build + save one document	Interview.create, Payment.create, PlacementTest.create
find(filter)	array of matches	getMyInterviews, getMyPlacementTests
findOne(filter)	single match or null	googleAuth (email), verifyPayment (order id)
findById(id)	single by _id	getCurrentUser, report endpoints
doc.save()	persist changes, validate	submitAnswer, finishInterview, savePlacementTest
findByIdAndUpdate(id, upd, { new: true })	one-call update, returns updated	verifyPayment with $inc credits

GAPS TO NAME BEFORE YOU ARE ASKED

- Ownership: savePlacementTest and getInterviewReport never compare record.userId with req.userId (IDOR).
- Money: createOrder trusts amount and credits from the request body instead of a server-side plan table.
- Duplicate email returns 500; the conventional answer is 409.
- AI scores are stored without validation or clamping to 0–10 in submitAnswer.
- Malformed ObjectIds become 500 (CastError) instead of 400.
- Create + credit deduction are two writes with no transaction or atomic guard.

Feature card & last-minute answers

The six product flows, then the answers a reviewer expects to hear.

INTERVIEW FLOW

```
POST /api/interview/resume           PDF -> pdfjs-dist text -> askAi -> role/experience/projects/skills
POST /api/interview/generate-questions  -50 credits, Interview.create, returns interviewId + questions
POST /api/interview/submit-answer    { interviewId, questionIndex, answer, timeTaken } -> AI scores
POST /api/interview/finish           averages -> finalScore + status 'completed'
GET  /api/interview/report/:id       saved report; page reads useParams()
```

PLACEMENT DRIVE

```
start (50 credits, testId) -> generate x5 (mcq_technical 12|written 4|aptitude 10|
soft_skills 5|project 4) -> evaluate (mcq local, others AI) -> final-report -> save
-> /placement-results/:id  -> GET /api/placement/report/:id (ownership checked)
```

AI SERVICE CONTRACT

askAi(messages, options) posts to Groq. options: json, returnParsed, temperature, max_tokens. response_format json_object is forced when json is true or any message contains 'json'. extractJson recovers raw JSON, fenced JSON or the first {...}/[...] region. HTTP 429 waits and retries up to 3 times. Generation validates the returned array; evaluation does not — AI output is external input.

ANSWERS TO THE FIFTEEN PRACTICE QUESTIONS

P4.1 app.use('/api/user', userRouter) mounts the router under the prefix — it does not connect to Mongo or verify a JWT.
P4.2 req.userId lets later handlers use the verified id without re-parsing the cookie or re-verifying.
P4.3 Valid token but no user document → the controller returns 404.
P4.4 401 is conventional for missing or invalid credentials.
P4.5 server/routes/user.route.js defines the middleware + controller sequence (store.js is Redux, the model is a schema).
P5.1 connectDb uses MONGODB_URL.
P5.2 timestamps: true adds createdAt and updatedAt.
P5.3 PlacementTest.create(...) creates the document.
P5.4 No — unique is an index; duplicates raise a MongoDB duplicate-key error (11000).
P5.5 Frontend checks can be bypassed; the API can be called directly.
P6.1 askAi() sends the prompts.
P6.2 A PDF uploaded in the resume field (multipart, upload.single('resume')).
P6.3 activeTestId identifies the attempt across save and the results URL.
P6.4 It navigates to /report/:id using the interview document id, then GET /api/interview/report/:id.
P6.5 No — inspect imports and network calls; CompanyPrep and Roadmap are static.

STATIC VERSUS API-DRIVEN PAGES

Page	Source of its content
Guidance.jsx · ResumeTips.jsx	POST /api/resume/guidance · POST /api/resume/ats-check (AI)
PlacementTestPage.jsx · PlacementResultsPage.jsx	questions + evaluate + final-report + save; then GET report/:id
InterviewHistory.jsx · InterviewReport.jsx	GET /api/interview/get-interview · GET report/:id
CompanyPrep.jsx · Roadmap.jsx	local arrays and JSX only — no axios, no fetch

DEMO SCRIPT IN SIX BEATS

- Start the server on 8000 (or point VITE_API_URL at the chosen port).
- Log in: POST /api/auth/google returns 200 and Set-Cookie: token.
- Reload: GET /api/user/current-user carries the cookie and answers 200 — Task 18 in one screenshot.
- Start an interview: generate-questions returns interviewId and questions; credits drop by 50.
- Answer once, then finish: submit-answer returns feedback; finish writes finalScore and completed.
- Close on a finding: the missing ownership check, with file and the 403 fix.

CLOSE WITH ONE IMPROVEMENT, NOT A LIST

Pick the ownership check: savePlacementTest and getInterviewReport should compare record.userId with req.userId and return 403 — the pattern already exists in getPlacementTestReport. Say the finding, the file, and the smallest correct fix, then stop.

If you remember one sentence:
"A request is a one-way pipeline: CORS, parsers, router, isAuth, controller/model, one HTTP response."